Maharashtra institute of technology CSN

(An autonomus institute)

Members name

PiyushKumar Ghate (A-25)

Upadesh Janjal(A-40)

Himanshu Kalambe (A-42)

Mini Project Report

Deep Reinforcement Learning vs Convolutional Neural Networks

for Image Classification on CIFAR-10

1. Abstract

This project investigates and compares two fundamentally different paradigms of machine learning — Convolutional Neural Networks (CNN) trained via supervised learning, and a Deep Q-Network (DQN) agent trained via Reinforcement Learning (RL) — applied to the CIFAR-10 image classification benchmark. We implement both approaches from scratch using TensorFlow/Keras, train them under comparable conditions, and evaluate them on accuracy, training stability, and computational cost. Our results demonstrate that CNNs are significantly more suited to static classification tasks, while RL-based agents offer flexibility for sequential decision-making scenarios.

2. Introduction

Deep Learning has transformed fields ranging from computer vision to natural language processing. Two prominent paradigms are Supervised Deep Learning — where models learn from labeled examples — and Reinforcement Learning (RL) — where agents learn optimal behaviors by interacting with an environment and receiving reward signals.

Convolutional Neural Networks (CNNs) have become the backbone of modern computer vision. Their hierarchical feature extraction through convolution operations makes them highly effective for image understanding tasks. In contrast, Deep Q-Networks (DQNs) were famously used by DeepMind to achieve superhuman performance on Atari games by learning solely from pixel observations and game scores.

This project models image classification as a Reinforcement Learning problem, treating each image as a 'state' and each class prediction as an 'action', with correct predictions yielding positive reward. This setup allows a direct performance comparison between the two approaches on the same dataset.

3. Dataset: CIFAR-10

CIFAR-10 is a widely used benchmark dataset in computer vision research. It consists of 60,000 colour images across 10 classes, split into 50,000 training and 10,000 test images. Each image is 32×32 pixels with 3 colour channels (RGB).

Property	Detail
Total Images	60,000
Training Set	50,000 images
Test Set	10,000 images
Image Dimensions	32 × 32 × 3 (RGB)
Number of Classes	10
Classes	Airplane, Auto, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck
Normalization	Pixel values scaled to [0, 1]

4. Methodology

4.1 CNN Architecture

The CNN model follows a VGG-inspired architecture with two convolutional blocks, each containing two Conv2D layers, Batch Normalization, Max Pooling, and Dropout. The final classification head uses a Dense(256) layer followed by a Softmax output layer with 10 units.

Layer Block	Components	Output Shape

Input	Raw 32×32 RGB image	32 × 32 × 3
Conv Block 1	Conv2D(32) × 2, BN, MaxPool, Dropout(0.25)	16 × 16 × 32
Conv Block 2	Conv2D(64) × 2, BN, MaxPool, Dropout(0.25)	8 × 8 × 64
Dense Head	Flatten → Dense(256) → BN → Dropout(0.5)	256
Output	Dense(10, softmax)	10

The CNN is compiled with the Adam optimizer (lr=0.001) and categorical crossentropy loss. Early stopping (patience=5) and learning rate reduction on plateau are used as callbacks to prevent overfitting and improve convergence.

4.2 Deep Q-Network (DQN) Architecture

The DQN agent models image classification as a Markov Decision Process (MDP):

- State (s): Flattened image pixel vector of size 3072
- Action (a): One of 10 class indices (0–9)
- Reward (r): +1 if prediction is correct, −1 otherwise
- Episode: Each image is treated as an independent episode

The Q-network is a fully-connected network: Input(3072) → Dense(512, ReLU) → Dropout(0.3) → Dense(256, ReLU) → Dropout(0.3) → Dense(10, linear). A separate target network with identical architecture is maintained for stable Q-value targets, and its weights are synced every 500 steps.

4.3 DQN Training Procedure

Key hyperparameters and techniques used in DQN training:

HYPERPARAMETER	VALUE
TOTAL TRAINING STEPS	20,000
REPLAY BUFFER CAPACITY	15,000 transitions
MINIBATCH SIZE	64
DISCOUNT FACTOR (Γ)	0.95
INITIAL EXPLORATION (E)	1.0
FINAL EXPLORATION (E MIN)	0.05
EPSILON DECAY RATE	0.9995 per step
Target Network Sync	Every 500 steps
Optimizer	Adam (lr=0.001)
Loss Function	Mean Squared Error (Bellman)

5. Results and Comparison

Both models were evaluated on the full CIFAR-10 test set (10,000 images). The table below summarizes the comparative performance metrics:

Metric	CNN (Supervised)	RL Agent (DQN)
Test Accuracy	~75–80%	~30–40%
Training Time	~20–40 minutes	~15–30 minutes
Learning Paradigm	Supervised	Reinforcement
Label Requirement	Yes (all samples)	No (uses reward only)
Convergence	Fast and stable	Slow and noisy
Best Use Case	Image classification	Sequential decision-making
Memory Requirement	Moderate	Higher (replay buffer)

5.1 CNN Performance Analysis

The CNN model converges steadily over 30 epochs, achieving approximately 75–80% test accuracy. The training and validation curves track closely, indicating minimal overfitting. Batch normalization and dropout effectively regulate the model. Per-class analysis shows that visually similar classes (cat/dog, automobile/truck) tend to have the most confusion, which is expected.

5.2 RL Agent Performance Analysis

The DQN agent begins with random actions (ε=1.0) and gradually shifts to learned policy actions. Despite 20,000 training steps, the agent achieves only 30–40% accuracy. This is expected: image classification is a fundamentally supervised task, and RL is at a disadvantage because it receives only a binary reward signal rather than gradient information from true labels. The reward curve shows gradual improvement, confirming that the agent is learning.

6. Discussion

The results clearly illustrate the domain-specificity of each learning paradigm:

- CNNs excel at image classification because they directly minimize a well-defined classificationloss using rich gradient information from all labeled samples.
- DQN agents are not inherently suited to static classification but shine in environments withsequential state transitions — games, robotics, autonomous driving.
- The combination of both (e.g., using CNN as the state encoder inside a DQN) is standard inmodern deep RL (e.g., Atari DQN by DeepMind) and would yield significantly better results.
- Sample efficiency is a key challenge for RL: it requires many more interactions than supervisedlearning to reach comparable performance.

One important insight is that RL is not a replacement for supervised learning in tasks with available labels — it is a complementary tool for settings where explicit supervision is unavailable or the task is inherently sequential.

7. Conclusion

This mini project successfully demonstrates and compares two major deep learning paradigms — CNN-based supervised learning and DQN-based reinforcement learning — on the CIFAR-10 image classification benchmark.

CNNs significantly outperform the RL agent (75–80% vs 30–40%) for this task, confirming that supervised learning is the preferred approach when labeled data is available.

However, RL offers a unique advantage when the task involves sequential decisions, exploration under uncertainty, or environments without pre-labeled data. Future work could explore using CNN feature extractors inside the RL agent, or applying RL to tasks like resource scheduling or game-playing where it truly excels.

8. Appendix — Code Structure Overview

The submitted .ipynb file is organized into the following sections:

Section 1	Library Imports and Environment Setup
Section 2	Global Seed Fixing for Reproducibility
Section 3	fetch_and_prepare_data() — Load and normalize CIFAR-10
Section 4	display_sample_grid() — Visualize training images
Section 5	construct_cnn_architecture() — Build CNN model
Section 6	compile_and_train_cnn() — Train with callbacks
Section 7	visualize_training_progress() — Plot accuracy/loss curves
Section 8	evaluate_classifier() — CNN test evaluation + confusion matrix
Section 9	build_dqn_network() — Construct Q-network
Section 10	ReplayMemoryBuffer class — Experience replay
Section 11	execute_dqn_training() — Full DQN training loop
Section 12	assess_rl_agent() — RL agent test evaluation
Section 13	plot_rl_learning_curves() — Reward and accuracy plots
Section 14	produce_comparison_chart() — Final CNN vs RL comparison
Section 15	Findings and Conclusion printout

GIT HUB link:- https://github.com/HRK98it/ANN-DL/tree/main/TA/TA2